

Homomorphic Time-Lock Puzzles

Walk-through of Malavolta & Thyagarajan ([eprint 2019/635](#)) and demo

This notebook is a **implementation** of the two HTLP schemes from the paper:

Scheme	Section	Homomorphism	We use it for
LHTLP	§4.1	linear: sum of secrets mod N	E-voting tally (<code>/vote</code>)
MHTLP	§4.2	multiplicative; with ± 1 encoding $\Rightarrow$ XOR	Multi-party coin flip (<code>/flip</code>)

1. Common helpers

Two utility functions that both schemes will use later.

1.1 `generate_strong_prime` — safe-prime sampling

A safe prime is $p = 2 \cdot p' + 1$ with p' also prime.

```
In [23]: import random
import time
from sympy import isprime, randprime

def generate_strong_prime(bits: int):
    '''Sample a safe prime  $p = 2 \cdot p' + 1$  ( $p'$  also prime). Returns (p, p').'''
    while True:
        p_prime = randprime(2 ** (bits - 2), 2 ** (bits - 1))
        p = 2 * p_prime + 1
        if isprime(p):
            return p, p_prime

# Quick sanity check: a 32-bit safe prime.
p, p_prime = generate_strong_prime(32)
print(f'p = {p}')
print(f'p' = (p-1)/2 = {p_prime} prime? {isprime(p_prime)}')
```

```
p = 3889512503
p' = (p-1)/2 = 1944756251 prime? True
```

1.2 `calibrate_T` — picking `T` for a wall-clock target

This is *not* part of the cryptographic scheme. Just measures how many modular squarings/sec the CPU manages, then returns $T = \text{squarings_per_sec} \cdot \text{target_seconds}$.

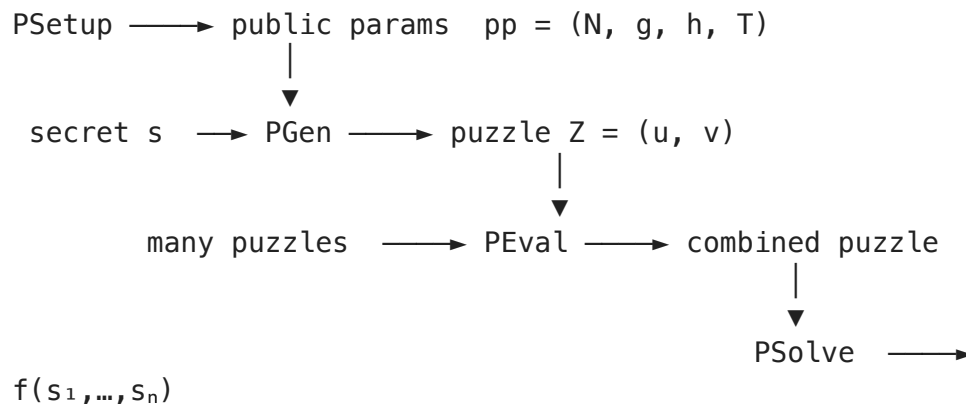
```
In [24]: def calibrate_T(target_seconds: float, bits: int = 128) -> int:
    '''Estimate squarings/sec, then scale to T = sq/sec · target_seconds.'''
    test_N = generate_strong_prime(bits)[0] * generate_strong_prime(bits)[0]
    x = random.randrange(2, test_N)
    squarings, start = 0, time.time()
    while time.time() - start < 1.0:
        x = pow(x, 2, test_N)          # one squaring = one tick of the clock
        squarings += 1
    return int(squarings * target_seconds)

squarings_per_sec = calibrate_T(1)
print(f'This CPU does about {squarings_per_sec:,} squarings/sec.')
print(f'For ~30 s of solving: T ≈ {30 * squarings_per_sec:,}')
```

This CPU does about 859,154 squarings/sec.
For ~30 s of solving: $T \approx 25,774,620$.

2. The Linearly-Homomorphic Time-Lock Puzzle

The data flow we are building:



2.1 PSetup (public parameters)

The trusted authority generates the public parameters once:

```
In [25]: def lhtlp_setup(bits: int, T: int):
    '''Generate public parameters pp = (N, g, h, T) for LHTLP.

    Returns a dict.'''
    # Step 1: two safe primes -> strong RSA modulus.
    p, _ = generate_strong_prime(bits)
    q, _ = generate_strong_prime(bits)
    N = p * q
    N2 = N * N
```

```

# The trapdoor: order of  $J_N = \phi(N)/2 = 2 \cdot p' \cdot q'$ . Kept secret by setup au
order_JN = ((p - 1) * (q - 1)) // 2

# Step 2: create a generator of  $J_N$ 
while True:
    g_tilde = random.randrange(2, N)
    g = (-pow(g_tilde, 2, N)) % N
    if g > 1:
        break

# Step 3: fast  $h = g^{(2^T)} \bmod N$  using the trapdoor.
# Anyone without order_JN must do  $T$  squarings instead – that's the time-
h = pow(g, pow(2, T, order_JN), N)

return {'N': N, 'N2': N2, 'g': g, 'h': h, 'T': T}

# Tiny parameters example:
pp = lhtlp_setup(bits=16, T=10)
print(f"N = {pp['N']} ({pp['N'].bit_length()} bits)")
print(f"g = {pp['g']}")
print(f"h = {pp['h']} (=  $g^{(2^T)} \bmod N$ )")
print(f"T = {pp['T']}")

```

```

N = 1409711521    (31 bits)
g = 956920413
h = 715547150    (=  $g^{(2^T)} \bmod N$ )
T = 10

```

2.2 PGen (secret encryption)

Given $s \in \mathbb{Z}_N$ and public params `pp`:

1. Sample fresh randomness $r \leftarrow \{1, \dots, N^2\}$.
2. $u := g^r \bmod N$.
3. $v := h^{(r \cdot N)} \cdot (1+N)^s \bmod N^2$.

```

In [26]: def lhtlp_pgen(pp, s: int):
    '''Encrypt secret s into a puzzle Z = (u, v).'''
    N, N2, g, h = pp['N'], pp['N2'], pp['g'], pp['h']

    r = random.randrange(1, N2)                # per-puzzle randomness – fresh r
    u = pow(g, r, N)                            #  $u = g^r \bmod N$  – PSolve
    blinding = pow(h, r * N, N2)                #  $h^{(r \cdot N)} \bmod N^2$  – hides (
    payload = pow(1 + N, s, N2)                 #  $(1+N)^s \bmod N^2$  – encoded
    v = (blinding * payload) % N2               #  $v = h^{(rN)} \cdot (1+N)^s$  – masks

    return (u, v)                               # puzzle publication

Z1 = lhtlp_pgen(pp, 5)
Z2 = lhtlp_pgen(pp, 10)

```

```
print(f'Z1 (s=5) = (u={Z1[0]}, v={Z1[1]})')
print(f'Z2 (s=10) = (u={Z2[0]}, v={Z2[1]})')
```

Z1 (s=5) = (u=1129366424, v=594969659570318347)

Z2 (s=10) = (u=539719287, v=996442688415749473)

2.3 PEval (summing puzzles homomorphically)

```
In [27]: # puzzles =
def lhtlp_peval(pp, puzzles):
    '''Combine puzzles homomorphically. Output is a fresh puzzle for  $\Sigma s_i$ .'
    N, N2 = pp['N'], pp['N2']
    u_tilde, v_tilde = 1, 1
    for u, v in puzzles: # (u = randomness, v = masked secret)
        u_tilde = (u_tilde * u) % N
        v_tilde = (v_tilde * v) % N2
    return (u_tilde, v_tilde) # return of combined puzzle

Z_sum = lhtlp_peval(pp, [Z1, Z2])
print(f'Combined puzzle for s = 5 + 10 : {Z_sum}')
```

Combined puzzle for s = 5 + 10 : (143064595, 212446605533143727)

2.4 PSoIve — the slow step

Three sub-steps:

1. **w := u^{(2^T) mod N}** — the *only* slow part. T sequential squarings; no shortcut without $\phi(N)$.
2. **Undo the blinding.** Note $w = (g^r)^{(2^T)} = (g^{(2^T)})^r = h^r$, so $w^N = h^{(rN)}$. Compute $(v \cdot (w^N)^{-1}) \bmod N^2$ to expose $(1+N)^s \bmod N^2$.
3. **Paillier discrete log.** Because $(1+N)^s \equiv 1 + sN \bmod N^2$, the secret is just $s = (\text{blob} - 1) / N$.

```
In [28]: def lhtlp_solve(pp, puzzle):
    '''Recover the secret from a puzzle. This is the slow op (T squarings).'''
    N, N2, T = pp['N'], pp['N2'], pp['T']
    u, v = puzzle

    # Step 1: T sequential squarings.
    w = u
    for _ in range(T):
        w = pow(w, 2, N)

    # Step 2: divide v by w^N inside Z*_{N^2}.
    wN = pow(w, N, N2) # reconstructs blinding pad of PGen
    wN_inv = pow(wN, -1, N2) # mod inverse
    blob = (v * wN_inv) % N2 # equals (1+N)^s mod N^2 -> mult v by mod inv

    # Step 3: discrete log.
    return (blob - 1) // N
```

```

recovered = lhttp_solve(pp, Z_sum)
print(f'PSolve(Z_sum) = {recovered} ← expected 15')
assert recovered == 15

```

PSolve(Z_sum) = 15 ← expected 15

2.5 PSoIve in detail

Same call as above, but printing every intermediatd.

```

In [29]: u, v = Z_sum                                     # unpacking the combined puzzle: u c
N, N2, T = pp['N'], pp['N2'], pp['T']                    # grabing the moduli and the time-lc

# Step 1 – redo the time-lock: square u exactly T times to reach h^r.
w = u
for _ in range(T):                                     # what actually takes time
    w = pow(w, 2, N)                                    # w ← w^2 mod N
print(f'w = u^(2^T) mod N = {w}')

# Step 2 – rebuild the blinding pad and invert it so we can cancel it out of
wN = pow(w, N, N2)
wN_inv = pow(wN, -1, N2)                               # mod inverse
print(f'w^N mod N^2 = {wN}')

# Step 3 – divide v by the pad → out pops (1+N)^s
blob = (v * wN_inv) % N2
direct = pow(1 + N, 15, N2)                             # Sanity check
print(f'(v / w^N) mod N^2 = {blob}')
print(f'(1+N)^15 mod N^2 = {direct}')
assert blob == direct

s = (blob - 1) // N
print(f's = (blob - 1)/N = {s}')                        # reovered secret
assert s == 15

```

```

w = u^(2^T) mod N = 837409942
w^N mod N^2 = 1383974932814290028
(v / w^N) mod N^2 = 21145672816
(1+N)^15 mod N^2 = 21145672816
s = (blob - 1)/N = 15

```

2.6 Four functions into a class

The demo (Demo/http.py) exposes LHTLP as a class so it can hold the public params as state. The class is just a thin wrapper around the four functions above

```

In [30]: class LHTLP:
    '''Thin wrapper matching the API used in Demo/http.py.'''

    def __init__(self, bits: int, T: int):
        self.pp = lhttp_setup(bits, T)

```

```

# Expose the parts the demo's other layers expect:
@property
def N(self): return self.pp['N']
@property
def T(self): return self.pp['T']

def PGen(self, s): return lhtlp_pgen(self.pp, s)
def PEval(self, puzzles): return lhtlp_peval(self.pp, puzzles)
def PSolve(self, puzzle): return lhtlp_solve(self.pp, puzzle)

# Quick check the wrapper agrees with the functional version.
toy = LHTLP(bits=16, T=10)
ZA = toy.PGen(7)
ZB = toy.PGen(35)
assert toy.PSolve(toy.PEval([ZA, ZB])) == 42
print('class-wrapped LHTLP works')

```

class-wrapped LHTLP works

3. The Multiplicatively-Homomorphic Time-Lock Puzzle (§4.2)

3.1 PSetup (identical to LHTLP)

```

In [31]: def mhtlp_setup(bits: int, T: int):
    p, _ = generate_strong_prime(bits)
    q, _ = generate_strong_prime(bits)
    N = p * q
    order_JN = ((p - 1) * (q - 1)) // 2
    while True:
        g_tilde = random.randrange(2, N)
        g = (-pow(g_tilde, 2, N)) % N
        if g > 1:
            break
    h = pow(g, pow(2, T, order_JN), N)
    return {'N': N, 'g': g, 'h': h, 'T': T}

pp_m = mhtlp_setup(bits=16, T=10)
print(f"N = {pp_m['N']} ({pp_m['N'].bit_length()} bits)")
print(f"g = {pp_m['g']}")
print(f"h = {pp_m['h']} (= g^(2^T) mod N)")
print(f"T = {pp_m['T']}")

```

```

N = 3916962781    (32 bits)
g = 656466403
h = 146248946    (= g^(2^T) mod N)
T = 10

```

3.2 PGen

```
In [32]: def mhtlp_pgen(pp, bit: int):
    assert bit in (0, 1)           # MHTLP only encodes a single bit
    N, g, h = pp['N'], pp['g'], pp['h']

    s = 1 if bit == 0 else N - 1   # encode bit as ±1: 0 → +1, 1 → -
    r = random.randrange(1, N * N) # same statistical-distance argument
    u = pow(g, r, N)                # commitment to r
    v = (pow(h, r, N) * s) % N      # secret masked by blinding pad h^r
    return (u, v)

for b in (0, 1):
    u, v = mhtlp_pgen(pp_m, b)     # build one puzzle per bit value
    encoded = '+1 (tails)' if b == 0 else 'N-1 ≡ -1 (heads)'
    print(f'bit={b} s={encoded:18s} puzzle u={u}, v={v}')
```

bit=0 s=+1 (tails) puzzle u=3905069803, v=2189823667
bit=1 s=N-1 ≡ -1 (heads) puzzle u=2537215410, v=1692620622

3.3 PEval (multiply puzzles = XOR the bits)

Identical structure to LHTLP's PEval, but everything is mod N (no N^2). The combined puzzle encodes the *product* of the secrets — which, with our ± 1 encoding, is the XOR of the bits.

```
In [33]: def mhtlp_peval(pp, puzzles):
    N = pp['N']
    u_tilde, v_tilde = 1, 1
    for u, v in puzzles:
        u_tilde = (u_tilde * u) % N
        v_tilde = (v_tilde * v) % N
    return (u_tilde, v_tilde)

bits = [1, 0, 1, 1] # example bits
puzzles = [mhtlp_pgen(pp_m, b) for b in bits]
combined = mhtlp_peval(pp_m, puzzles)
print(f'bits = {bits} → combined puzzle = {combined}')
```

bits = [1, 0, 1, 1] → combined puzzle = (2300091316, 1147863665)

3.4 PSoIve (XOR recovery)

```
In [34]: def mhtlp_solve(pp, puzzle):
    N, T = pp['N'], pp['T']
    u, v = puzzle

    # T sequential squarings, slow time-lock step.
    w = u
    for _ in range(T):             # T squarings, inherently sequential
        w = pow(w, 2, N)          # after the loop: w = u^(2^T) = h^r mod N

    # Undo the blinding, for MHTLP this is just modular division.
```

```
s = (v * pow(w, -1, N)) % N
return 0 if s == 1 else 1
```

```
outcome = mhtlp_solve(pp_m, combined)
expected = bits[0] ^ bits[1] ^ bits[2] ^ bits[3] # ground truth
print(f'XOR ground truth = {expected}')
print(f'PSolve gives = {outcome}')
assert outcome == expected
```

```
XOR ground truth = 1
PSolve gives = 1
```

3.5 Wrap into the class the demo uses

```
In [35]: class MHTLP:
    def __init__(self, bits: int, T: int):
        self.pp = mhtlp_setup(bits, T)

    @property
    def N(self): return self.pp['N']
    @property
    def T(self): return self.pp['T']

    def PGen(self, bit): return mhtlp_pgen(self.pp, bit)
    def PEval(self, puzzles): return mhtlp_peval(self.pp, puzzles)
    def PSolve(self, puzzle): return mhtlp_solve(self.pp, puzzle)

m = MHTLP(bits=16, T=10)
assert m.PSolve(m.PEval([m.PGen(1), m.PGen(1), m.PGen(0)])) == 0 # 1⊕1⊕0
print('class-wrapped MHTLP works ✓')
```

class-wrapped MHTLP works ✓

4. Application 1: e-voting (Codex vs Claude)

This mirrors the `/vote` flow in the live demo.

Protocol (paper §6.1, specialised to $m = 2$ candidates)

Setup (trusted authority, one-time): publish `pp` from `lhtlp_setup`.

Voting phase. Each voter V_i picks a candidate and posts a 2-tuple of LHTLP puzzles, e.g. `(PGen(1), PGen(0))` for Codex.

Counting phase. The tallier `PEval` s each column and runs `PSolve` **twice** — work is independent of the number of voters.

4.1 Set up the election and cast ballots


```
In [36]: election = LHTLP(bits=64, T=200) # small params so the notebook is snappy

voter_choices = ['Codex', 'Codex', 'Claude', 'Codex', 'Claude', 'Codex', 'Co

ballots = []
for choice in voter_choices:
    if choice == 'Codex':
        ballots.append((election.PGen(1), election.PGen(0))) # (codex, cla
    else:
        ballots.append((election.PGen(0), election.PGen(1)))

print(f'{len(ballots)} ballots cast – each is a pair of LHTLP puzzles.')
print(f'One puzzle component has ~{(election.N * election.N).bit_length()} b
```

7 ballots cast – each is a pair of LHTLP puzzles.
One puzzle component has ~256 bits.

4.2 Tally homomorphically, then solve twice

```
In [37]: codex_puzzles = [b[0] for b in ballots]
claude_puzzles = [b[1] for b in ballots]

t0 = time.time()
codex_combined = election.PEval(codex_puzzles)
claude_combined = election.PEval(claude_puzzles)
print(f'PEval over {len(ballots)} ballots: {(time.time()-t0)*1000:.2f} ms –

t0 = time.time()
codex_total = election.PSolve(codex_combined)
claude_total = election.PSolve(claude_combined)
print(f'2× PSolve (T={election.T}): {(time.time()-t0)*1000:.2f} ms.\n')

print(f'Result: Codex = {codex_total}, Claude = {claude_total}')
print(f'Truth : Codex = {voter_choices.count("Codex")}, Claude = {voter_ch
assert codex_total == voter_choices.count('Codex')
assert claude_total == voter_choices.count('Claude')
```

PEval over 7 ballots: 0.55 ms – independent of T.
2× PSolve (T=200): 1.19 ms.

Result: Codex = 5, Claude = 2
Truth : Codex = 5, Claude = 2

5. Application 2: multi-party coin flip

This mirrors the `/flip` flow. The XOR variant noted at the end of §4.2 is what we run, and what's shown below.

Protocol.

1. Setup publishes `pp` from `mhtlp_setup`.
2. Each participant P_i draws $b_i \leftarrow \{0,1\}$ and broadcasts $Z_i = \text{mhtlp_pgen}(pp, b_i)$.

3. Anyone can combine via `PEval`. Solving once reveals $b = b_1 \oplus \dots \oplus b_n$.

Bias resistance. If $n-1$ parties collude, they can choose bits after seeing the others' puzzles, but they cannot see the honest party's bit before committing (it's behind the time-lock). XOR with a fresh uniform bit is uniform, so the coin stays fair.

5.1 Five participants, each broadcasts a random time-locked bit

```
In [38]: flip = MHTLP(bits=64, T=200)

participants = 5
bits = [random.randint(0, 1) for _ in range(participants)]
puzzles = [flip.PGen(b) for b in bits]

print('Participant bits (hidden behind time-locks):', bits)
print('(In a real run, the bits would not be visible until PSolve completes.)')
```

```
Participant bits (hidden behind time-locks): [0, 1, 0, 0, 0]
(In a real run, the bits would not be visible until PSolve completes.)
```

5.2 Combine and solve once

```
In [39]: combined = flip.PEval(puzzles)
outcome = flip.PSolve(combined)

expected = 0
for b in bits:
    expected ^= b

face = 'heads' if outcome == 1 else 'tails'
print(f'Coin flip result : {outcome} ({face})')
print(f'XOR ground truth : {expected}')
assert outcome == expected
```

```
Coin flip result : 1 (heads)
XOR ground truth : 1
```

5.3 Client-side puzzle generation in the live demo

In the `/flip` demo each participant's puzzle is generated entirely inside their **browser tab** — the server never sees the bit. The steps map directly onto §3.2:

Step	Python (this notebook)	Browser (<code>htlp.js</code>)
public params	<code>mhtlp_setup</code> returns <code>pp</code>	<code>GET /flip-params</code> → <code>{ N, g, h, T }</code>
random bit	<code>random.randint(0, 1)</code>	<code>crypto.getRandomValues</code> (browser CSPRNG)
puzzle	<code>mhtlp_pgen(pp, bit)</code>	<code>mhtlpPGen(pp, bit)</code> — same arithmetic, <code>BigInt</code>

Step	Python (this notebook)	Browser (<code>http.js</code>)
server receives	—	POST /flip { u, v } — bit not included

`PSolve` runs server-side only after the collection phase closes.

6. Worked examples with tiny numbers

The cells above showed the algebra at production scale. The two cells below redo each application **end-to-end with deliberately small parameters**, printing every intermediate so you can verify by hand that:

- each `PGen` produces a sensible (u, v) ,
- `PEval`'s combined $(\tilde{u}, \tilde{v})$ really is the component-wise product mod the right modulus,
- `PSolve` recovers exactly the secret you'd expect from the homomorphism.

Parameters are kept tiny on purpose (`bits=16` , `T=5`) so every integer fits on one line.

6.1 Application 1: e-voting, every intermediate printed

```
In [ ]: import random

# ----- 1. Setup (tiny parameters) -----
random.seed(0)                                # deterministic output
demo_pp = lhttp_setup(bits=16, T=5)            # T=5 so we can print
N, N2, g, h, T = demo_pp['N'], demo_pp['N2'], demo_pp['g'], demo_pp['h'], de
print('--- public parameters -----')
print(f'N = {N}    ({N.bit_length()} bits)')
print(f'N² = {N2}')
print(f'g = {g}')
print(f'h = g^(2^T) mod N = {h}')
print(f'T = {T}\n')

# ----- 2. Three voters cast their ballots -----
choices = ['Codex', 'Claude', 'Codex']        # ground truth: 2 vs
expected = (choices.count('Codex'), choices.count('Claude'))

def pgen_verbose(s, label):
    '''lhttp_pgen but printing every intermediate.'''
    r = random.randrange(1, N2)
    u = pow(g, r, N)
    blinding = pow(h, r * N, N2)
    payload = pow(1 + N, s, N2)
    v = (blinding * payload) % N2
    print(f' {label}: s={s}  r={r}')
    print(f'          u = g^r mod N          = {u}')
    print(f'          blinding = h^(rN) % N² = {blinding}')
```

```

    print(f'          payload = (1+N)^s%N^2= {payload}')
    print(f'          v = blinding.payload = {v}')
    return (u, v)

ballots = []
for i, choice in enumerate(choices, start=1):
    print(f'---- ballot {i} (voter chose {choice}) -----')
    s_codex, s_claude = (1, 0) if choice == 'Codex' else (0, 1)
    Z_codex = pgen_verbose(s_codex, 'codex-puzzle ')
    Z_claude = pgen_verbose(s_claude, 'claude-puzzle ')
    ballots.append((Z_codex, Z_claude))
    print()

# ----- 3. Tally each column with PEval -----
def peval_verbose(puzzles, column_name):
    print(f'---- PEval over the {column_name} column -----')
    u_tilde, v_tilde = 1, 1
    for k, (u, v) in enumerate(puzzles, start=1):
        u_tilde = (u_tilde * u) % N
        v_tilde = (v_tilde * v) % N2
        print(f'   after ballot {k}:  ã = {u_tilde}   ã = {v_tilde}')
    print(f' combined puzzle: ({u_tilde}, {v_tilde})\n')
    return (u_tilde, v_tilde)

codex_combined = peval_verbose([b[0] for b in ballots], 'Codex')
claude_combined = peval_verbose([b[1] for b in ballots], 'Claude')

# ----- 4. PSolve each combined puzzle, step by step -----
def psolve_verbose(puzzle, label):
    print(f'---- PSolve on {label} -----')
    u, v = puzzle
    w = u
    print(f' start: w = u = {w}')
    for step in range(1, T + 1):
        w = pow(w, 2, N)
        print(f' after squaring #{step}: w = {w}')
    wN = pow(w, N, N2)
    wN_inv = pow(wN, -1, N2)
    blob = (v * wN_inv) % N2
    s = (blob - 1) // N
    print(f' w^N mod N^2 = {wN}')
    print(f' (w^N)^-1 mod N^2 = {wN_inv}')
    print(f' blob = v·wN_inv = {blob} (should equal (1+N)^S mod N^2)')
    print(f' check (1+N)^{s} %N^2= {pow(1 + N, s, N2)}')
    print(f' recovered S = (blob-1)//N = {s}\n')
    return s

codex_count = psolve_verbose(codex_combined, 'Codex column')
claude_count = psolve_verbose(claude_combined, 'Claude column')

# ----- 5. Final tally -----
print('=== FINAL TALLY ===')
print(f'Codex = {codex_count} (expected {expected[0]})')
print(f'Claude = {claude_count} (expected {expected[1]})')
assert (codex_count, claude_count) == expected
print('tally matches the ground truth')

```

```

--- public parameters -----
N = 2716597981 (32 bits)
N² = 7379904590373276361
g = 2421975149
h = g^(2^T) mod N = 779364023
T = 5

```

```

--- ballot 1 (voter chose Codex) -----
codex-puzzle : s=1 r=373402508605428822
    u = g^r mod N = 467226597
    blinding = h^(rN) %N²= 2293823403206403556
    payload = (1+N)^s %N²= 2716597982
    v = blinding·payload = 4597342164463586911
claude-puzzle: s=0 r=4481891911369680483
    u = g^r mod N = 1915152334
    blinding = h^(rN) %N²= 655810493037707733
    payload = (1+N)^s %N²= 1
    v = blinding·payload = 655810493037707733

```

```

--- ballot 2 (voter chose Claude) -----
codex-puzzle : s=0 r=3302422582397125125
    u = g^r mod N = 750585165
    blinding = h^(rN) %N²= 7077067162852041759
    payload = (1+N)^s %N²= 1
    v = blinding·payload = 7077067162852041759
claude-puzzle: s=1 r=2014778559842358804
    u = g^r mod N = 1941314670
    blinding = h^(rN) %N²= 4089382378220527547
    payload = (1+N)^s %N²= 2716597982
    v = blinding·payload = 6802207224669355008

```

```

--- ballot 3 (voter chose Codex) -----
codex-puzzle : s=1 r=1284573236628236407
    u = g^r mod N = 2585139024
    blinding = h^(rN) %N²= 2855693951134963940
    payload = (1+N)^s %N²= 2716597982
    v = blinding·payload = 3978564342892611415
claude-puzzle: s=0 r=1288927344666773108
    u = g^r mod N = 182877972
    blinding = h^(rN) %N²= 6831597244765940886
    payload = (1+N)^s %N²= 1
    v = blinding·payload = 6831597244765940886

```

```

--- PEval over the Codex column -----
after ballot 1: ũ = 467226597   ṽ = 4597342164463586911
after ballot 2: ũ = 1179479484   ṽ = 955489026659339939
after ballot 3: ũ = 1245573632   ṽ = 2101534694347413222
combined puzzle: (1245573632, 2101534694347413222)

```

```

--- PEval over the Claude column -----
after ballot 1: ũ = 1915152334   ṽ = 655810493037707733
after ballot 2: ũ = 2642285168   ṽ = 6119264239619297461
after ballot 3: ũ = 1488202490   ṽ = 3284245639776474297
combined puzzle: (1488202490, 3284245639776474297)

```

```

--- PSolve on Codex column -----

```

```

start: w = u = 1245573632
after squaring #1: w = 1899963344
after squaring #2: w = 1253137237
after squaring #3: w = 432388329
after squaring #4: w = 972113972
after squaring #5: w = 2118039317
w^N mod N^2      = 3907209288533849522
(w^N)^-1 mod N^2 = 5943150062656134479
blob = v·wN_inv   = 5433195963    (should equal (1+N)^S mod N^2)
check (1+N)^2 %N^2= 5433195963
recovered S       = (blob-1)//N = 2

--- PSolve on Claude column -----
start: w = u = 1488202490
after squaring #1: w = 1346986882
after squaring #2: w = 488269253
after squaring #3: w = 1267697583
after squaring #4: w = 639346949
after squaring #5: w = 677868769
w^N mod N^2      = 7141516503921150402
(w^N)^-1 mod N^2 = 4672614729223751625
blob = v·wN_inv   = 2716597982    (should equal (1+N)^S mod N^2)
check (1+N)^1 %N^2= 2716597982
recovered S       = (blob-1)//N = 1

=== FINAL TALLY =====
Codex = 2    (expected 2)
Claude = 1   (expected 1)
✓ tally matches the ground truth

```

6.2 Application: coin flip, every intermediate printed

```

In [ ]: import random

# ----- 1. Setup (tiny parameters) -----
random.seed(1)
demo_pp_m = mhtlp_setup(bits=16, T=5)
Nm, gm, hm, Tm = demo_pp_m['N'], demo_pp_m['g'], demo_pp_m['h'], demo_pp_m['T']
print('--- public parameters ---')
print(f'N = {Nm}    ({Nm.bit_length()} bits)')
print(f'g = {gm}')
print(f'h = g^(2^T) mod N = {hm}')
print(f'T = {Tm}\n')

# ----- 2. Four participants commit to bits -----
bits_truth = [1, 0, 1, 1] # ground-truth XOR = 1
expected_xor = 0
for b in bits_truth:
    expected_xor ^= b

def mhtlp_pgen_verbose(bit, who):
    s = 1 if bit == 0 else Nm - 1 # ±1 encoding
    r = random.randrange(1, Nm * Nm)
    u = pow(gm, r, Nm)
    blinding = pow(hm, r, Nm) # h^r - the mask

```

```

    v = (blinding * s) % Nm
    sign = '+1' if s == 1 else 'N-1 ( $\equiv -1$ )'
    print(f' {who}: bit={bit} s={sign} r={r}')
    print(f'          u = g^r mod N      = {u}')
    print(f'          h^r mod N (mask)    = {blinding}')
    print(f'          v = h^r * s mod N    = {v}')
    return (u, v)

puzzles = []
for i, b in enumerate(bits_truth, start=1):
    print(f'--- participant {i} -----')
    puzzles.append(mhtmlp_pgen_verbose(b, f'P{i}'))
    print()

# ----- 3. PEval - multiply the puzzles -----
print('--- PEval (multiplicative fold over  $Z*_N$ ) -----')
u_tilde, v_tilde = 1, 1
for k, (u, v) in enumerate(puzzles, start=1):
    u_tilde = (u_tilde * u) % Nm
    v_tilde = (v_tilde * v) % Nm
    print(f' after P{k}:    $\tilde{u}$  = {u_tilde}    $\tilde{v}$  = {v_tilde}')
combined = (u_tilde, v_tilde)
print(f' combined puzzle: {combined}\n')

# ----- 4. PSolve step by step -----
print('--- PSolve on the combined puzzle -----')
u, v = combined
w = u
print(f' start: w = u = {w}')
for step in range(1, Tm + 1):
    w = pow(w, 2, Nm)
    print(f' after squaring #{step}: w = {w}')
w_inv = pow(w, -1, Nm)
s_recovered = (v * w_inv) % Nm
print(f' w^-1 mod N      = {w_inv}')
print(f' s = v * w^-1 mod N = {s_recovered}')

# ----- 5. Map  $\pm 1$  back to a bit -----
if s_recovered == 1:
    outcome = 0
    encoding = '+1  $\rightarrow$  bit 0 (tails)'
elif s_recovered == Nm - 1:
    outcome = 1
    encoding = 'N-1  $\equiv -1 \rightarrow$  bit 1 (heads)'
else:
    raise AssertionError('combined secret is not  $\pm 1$  - algebra broke')
print(f' decoded: {encoding}\n')

print('=== COIN-FLIP RESULT =====')
print(f'XOR ground truth = {bits_truth[0]}  $\oplus$  {bits_truth[1]}  $\oplus$  {bits_truth[2]}  $\oplus$  {bits_truth[3]}')
print(f'PSolve outcome    = {outcome}')
assert outcome == expected_xor
print('coin lands on the homomorphic XOR of all four bits')

```

```
--- public parameters -----
N = 1809778813    (31 bits)
g = 1444007172
h = g^(2^T) mod N = 79155629
T = 5
```

```
--- participant 1 -----
P1: bit=1  s=N-1 ( $\equiv -1$ )  r=1176298286020543698
      u = g^r mod N      = 1397543551
      h^r mod N (mask)   = 85352560
      v = h^r · s mod N  = 1724426253
```

```
--- participant 2 -----
P2: bit=0  s=+1  r=2284791064310049674
      u = g^r mod N      = 906144970
      h^r mod N (mask)   = 16656428
      v = h^r · s mod N  = 16656428
```

```
--- participant 3 -----
P3: bit=1  s=N-1 ( $\equiv -1$ )  r=2072911645936348997
      u = g^r mod N      = 89991394
      h^r mod N (mask)   = 410665486
      v = h^r · s mod N  = 1399113327
```

```
--- participant 4 -----
P4: bit=1  s=N-1 ( $\equiv -1$ )  r=3004942216765706338
      u = g^r mod N      = 972507023
      h^r mod N (mask)   = 1307075236
      v = h^r · s mod N  = 502703577
```

```
--- PEval (multiplicative fold over  $Z*_N$ ) -----
after P1:   $\tilde{u}$  = 1397543551   $\tilde{v}$  = 1724426253
after P2:   $\tilde{u}$  = 143243798    $\tilde{v}$  = 1166517657
after P3:   $\tilde{u}$  = 246628695    $\tilde{v}$  = 1750554348
after P4:   $\tilde{u}$  = 1467881871   $\tilde{v}$  = 520401794
combined puzzle: (1467881871, 520401794)
```

```
--- PSolve on the combined puzzle -----
start: w = u = 1467881871
after squaring #1: w = 1335558453
after squaring #2: w = 1229212031
after squaring #3: w = 1678539821
after squaring #4: w = 818739812
after squaring #5: w = 1289377019
w^-1 mod N      = 157620094
s = v · w^-1 mod N = 1809778812
decoded: N-1  $\equiv -1$  → bit 1 (heads)
```

```
=== COIN-FLIP RESULT =====
XOR ground truth = 1  $\oplus$  0  $\oplus$  1  $\oplus$  1 = 1
PSolve outcome   = 1
✓ coin lands on the homomorphic XOR of all four bits
```